Basic RL Algorithms

Applied Machine Learning — Session 4, Chapter 2

Two Approaches to RL

Model-Based: Learn a model of the environment, then plan → Sample efficient, but hard to build an accurate model

Model-Free: Learn directly from experience (no model) → More practical for complex environments

Today: Model-Free RL

- Value-based: learn $Q(s, a)$ → Q-Learning
- Policy-based: learn π directly → Policy Gradient

Q-Learning: The Idea

Build a table of Q(s, a) — the long-term value of each action in each state.

```
Q-Table (4×4 Grid, 4 actions):
         ← ↓ → ↑
State 0: [0.1, 0.3, 0.5, 0.0]
State 1: [0.0, 0.8, 0.2, 0.1]
...
State 15:[0.0, 0.0, 0.0, 0.0]  ← goal
```

Optimal policy: `a* = argmax_a Q(s, a)`

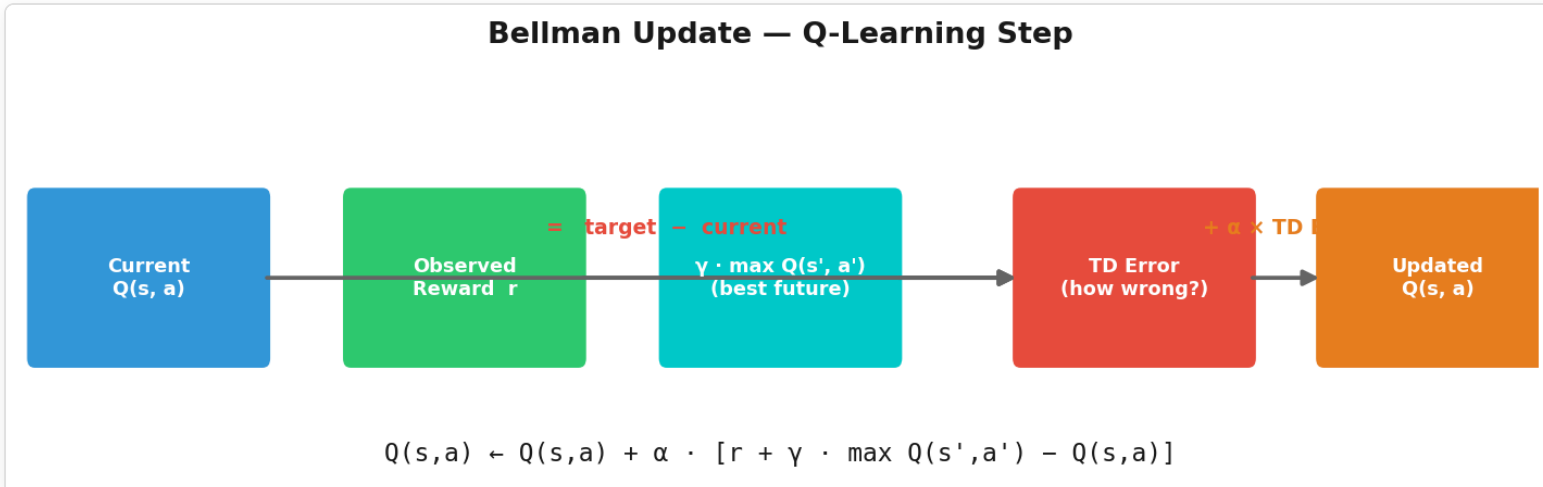
Learning: Update Q-values using observed experience.

The Bellman Equation

The core update rule of Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max Q(s', a') - Q(s, a)]$$

$$\underbrace{\qquad\qquad\qquad\qquad}_{\text{TD Error}}$$



TD Error Intuition

Like a GPS updating its arrival estimate:

```
GPS predicts: 30 min
Reality: road closed, +10 min
GPS update: now predicts 40 min

Q-Learning:
Current Q: 0.5
Observed reward: 0 + future value: 0.8
TD Error: 0.8 - 0.5 = +0.3
Updated Q: 0.5 + α * 0.3
```

The Q-Learning Algorithm

```
Initialize Q[states, actions] = 0

for episode in range(n_episodes):
    state = env.reset()
    done = False

    while not done:
        # ε-greedy action selection
        if random() < epsilon:
            action = env.action_space.sample()  # explore
        else:
            action = argmax(Q[state])            # exploit

        # Take action, observe outcome
        next_state, reward, done, _ = env.step(action)

        # Bellman update
        Q[state, action] += alpha * (
```

Hyperparameters

Parameter	Symbol	Typical value	Effect
Learning rate	α	0.1 – 0.5	How fast we update Q
Discount	γ	0.95 – 0.99	How much we value future
Initial epsilon	ε	0.9 – 1.0	Start with lots of exploration
Epsilon decay		0.99 – 0.999	Slow shift to exploitation
Min epsilon		0.01	Always keep a little exploration
Episodes		1,000 – 10,000	More = better learning

SARSA: The On-Policy Alternative

SARSA = State, Action, Reward, State, Action

```
Q(s, a) ← Q(s, a) + α · [r + γ · Q(s', a') - Q(s, a)]
```

Key difference: Uses the *actual next action taken*, not the *best possible*

	Q-Learning	SARSA
Type	Off-policy	On-policy
Updates with	max Q(s', a')	Q(s', a') actually taken
Risk	Can be dangerous during exploration	Safer — avoids risky paths

Policy Gradient (Conceptual)

Instead of Q-values → directly learn the policy $\pi(a|s)$

REINFORCE:

```
Collect trajectory: s₀, a₀, r₁, s₁, a₁, r₂, ...
For each step: increase probability of actions that led to high reward
               decrease probability of actions that led to low reward
```

```
# Pseudocode
loss = -sum(log_prob(actions) * returns)
optimizer.step(loss)
```

Modern algorithms: PPO, A3C, SAC — all extend this idea with neural networks.

The FrozenLake Environment

```
SFFF       S = Start  (0)
FHFH       F = Frozen (safe)
FFFH       H = Hole   (game over)
HFFG       G = Goal   (+1 reward)
```

- 16 states (cells 0–15)
- 4 actions (Left=0, Down=1, Right=2, Up=3)
- Reward: +1 at goal, 0 everywhere else
- Slippery ice: actions don’t always go as planned!

Why FrozenLake Is Hard

Problem 1 — Sparse rewards:

Only one reward (+1 at goal). All other transitions give 0.

The agent must explore many steps before seeing any signal.

Problem 2 — Stochasticity:

Even good actions may fail. The agent slips randomly.

→ Same action, same state → different outcome each time.

Problem 3 — Delayed credit:

The goal is far from the start.

Which earlier actions caused the success? (Credit assignment problem)

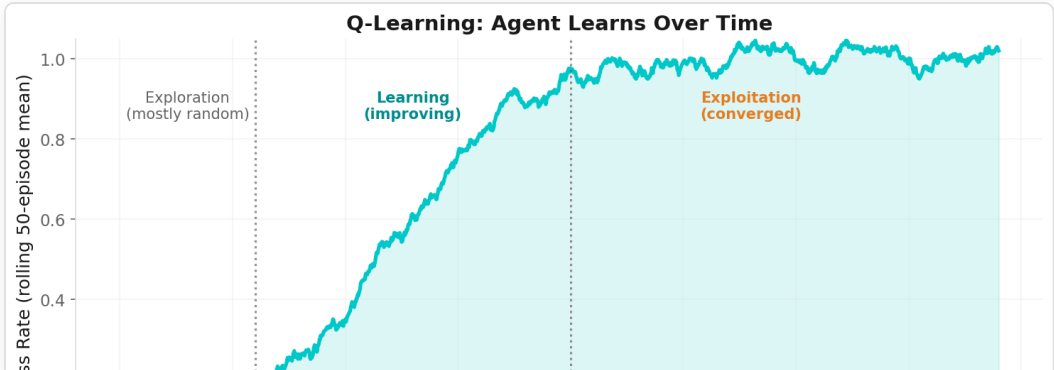
This is why RL is hard — and why Q-Learning with enough episodes works anyway.

Now: Live Example!

→ Open `02-examples/ch11_rl_algorithms_examples.ipynb`

We will:

1. Set up FrozenLake with Gymnasium
2. Implement Q-Learning step by step
3. Watch the agent improve over thousands of episodes
4. Visualize the learned Q-table



Now: Exercises!

→ Open `03-exercises/ch11_rl_algorithms_exercises.ipynb`

Task: Implement Q-Learning from scratch.

Train the agent, tune epsilon, visualize progress.

~10 minutes

Key Takeaways

- Q-Learning: off-policy, learns Q(s,a) via Bellman equation
- Bellman update: current estimate + α × TD error
- ε-greedy: start exploring, gradually exploit
- SARSA: on-policy, safer in risky environments
- Policy Gradient: learn the policy directly (for continuous actions)

Next: Chapter 12

Capstone: End-to-End ML Workflow

"Time to put everything together."